Modern AI Course

An abbreviated and unofficial script

Jörg Neulist

March 2026

Lecture 11: Self Attention

LLM setting

- Input is T tokens from a vocabulary of V words in one-hot encoding. $X_{\text{oh}} \in \mathbf{R}^{T \times V}$ and

$$\text{"the quick brown ..."} \mapsto X_{\text{oh}} = \begin{bmatrix} — & x_1^\intercal & — \\ — & x_2^\intercal & — \\ — & x_3^\intercal & — \\ & \vdots & \end{bmatrix} \begin{matrix} \text{"the"} \\ \text{"quick"} \\ \text{"brown"} \\ \vdots \end{matrix}$$

- We use embedding to reduce dimensions and enrich semantic interpretation, using $W_E \in \mathbb{R}^{d \times V}$:

$$X := X_{\text{oh}} W_E^\intercal \quad \in \mathbb{R}^{T \times d}$$

The self-attention operation

$$Y = \text{SelfAttention}(Q, K, V)$$

1. form three matrices $Q, K, V \in \mathbb{R}^{T \times d}$:

$$Q = XW_Q^\intercal, \quad K = XW_K^\intercal, \quad V = XW_V^\intercal$$

   where $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$.

2. Output self-attention operation:

$$Y = \text{SelfAttention}(Q, K, V) := \text{softmax}\left(\frac{QK^\intercal}{\sqrt{d}}\right) V$$

   (softmax applied by row)

3. The attention matrix $A := \text{softmax}\left(\frac{QK^\intercal}{\sqrt{d}}\right)$ can then be thought of as being applied to the input, because:

$$Y = \text{SelfAttention}(Q, K, V) = AXW_V^\intercal$$

4. One important conclusion is that self-attention *mixes* the input rows, which normal MLP doesn't do.

5. This can be interpreted as a soft dict lookup.

Self-attention in code

```
import math
import torch

def self_attention(Q, K, V):
    d = Q.shape[-1]
    return torch.softmax(Q @ K.transpose(-1, -2) / math.sqrt(d), dim=-1) @ V

X = torch.randn(5, 100, 10)
WQ = torch.randn(10, 10)
WK = torch.randn(10, 10)
WV = torch.randn(10, 10)
Q, K, V = X @ WQ.T, X @ WK.T, X @ WV.T
Y = self_attention(Q, K, V) # V has shape [5, 100, 10]
```

Properties of self-attention

- It doesn't matter how we order the rows of X.
- Every output row can depend on every input row (full mixing over rows).

Multihead Attention

Motivation: The attention operation is the same for every single row of the input:

$$A(X)XW_V^\intercal = \begin{bmatrix} — & a_1^\intercal & — \\ — & a_2^\intercal & — \\ & \vdots & \end{bmatrix} XW_V^\intercal = \begin{bmatrix} — & a_1^\intercal X & — \\ — & a_2^\intercal X & — \\ & \vdots & \end{bmatrix} W_V^\intercal$$

This is not optimal; we desire more adaptability.

1. Starting with $Q = XW_Q^\intercal$, $K = XW_K^\intercal$, $V = XW_V^\intercal$.

2. Partition each Q, K, V by columns in h groups called "heads":

$$Q =: \begin{bmatrix} | & | & & | \\ Q_1 & Q_2 & \dots & Q_h \\ | & | & & | \end{bmatrix}$$

   Each $Q_i \in \mathbb{R}^{T \times \frac{d}{h}}$. Split K and V in the same way

3. Define multihead attention as the independent operation of the separate attention blocks, projected by a final output projection $W_p \in \mathbb{R}^{d \times d}$

$$Y =: \begin{bmatrix} | & & | \\ \text{SelfAttention}(Q_1, K_1, V_1) & \dots & \text{SelfAttention}(Q_h, K_h, V_h) \\ | & & | \end{bmatrix} W_p$$

Lecture 12: Transformers

Transformer Layers and Transformers

- Transformer Layer = self-attention + two-layer MLP + normalisation + residual connections
- Transformer = Embedding + N times Transformer Layers + normalization + linear output layer
- Transformer Layer with input $X \in \mathbb{R}^{T \times d}$ (T is the sequence length, i.e. the number of tokens, d is the embedding dimensionality)
  1. Apply self-attention with normalisation of each token independently:

$$Z := X + \text{MultiheadAttention}(\text{norm}(X)), \text{ where}$$

$$\text{norm}(X) := \frac{X}{\|X\|_2}, \text{applied rowwise}$$

  2. Return

$$Y := Z + \text{MLP}(\text{norm}(Z)), \text{ where}$$

$$\text{MLP}(X) := \sigma(XW_1^\intercal)W_2^\intercal$$

     Typically, the hidden dimension of the MLP is about four times as large as in- and output, i.e. $W_1 \in \mathbb{R}^{d_{\text{MLP}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{MLP}}}$ and $d_{\text{MLP}} \approx 4d$

- Transformer Layer with input $X_{\text{oh}} \in \mathbb{R}^{T \times V}$ (V is the vocabulary size, used with one-hot encoding)
  1. Embed, to reduce dimensionality:

$$X_0 := X_{oh} W_E^\intercal, \text{ where } W_E \in \mathbb{R}^{d \times V}, \text{ so } X_0 \in \mathbb{R}^{T \times d}$$

2. Repeat N times:

$$X_i := \text{TransformerLayer}_i(X_{i-1})$$

   i.e. N different Transformer Layers (different MLP and attention weights)

3. Output:

$$Y := \text{norm}(X_N)W_0^\intercal, \text{ where } W_0 \in (R)^{V \times d}, \text{ so } Y \in (R)^{T \times V}$$

Parallel Prediction

- Prediction gives T tokens, not only one. Goal, when training a transformer, is to predict the next word from every prefix. Let h be the prediction mapping:

$$h : (R)^{T \times V} \to (R)^V, \text{ with the loss defined as}$$

$$L := \sum_{t=1}^{T-1} L_{\text{CE}}(h(X_{1:t}), Y_t),$$

  where Y_t is the $t+1$st word.

- We apply the Transformer to the entire input sequence:

$$\hat{Y} := \text{Transformer}(X_{oh}) \quad \in (R)^{T \times V}$$

$$L := \sum_{t=1}^{T} L_{\text{CE}}(\hat{Y}_t, Y_t)$$

  The obvious problem is that the model could just look ahead in the sequence to "predict" the ground truth. That is not desirable. Moreover, the Attention operations are mixing op the input sequence, so they are not order-aware with respect to the input text. This leads us to...

Positional Encoding / Embedding (fix invariance to word order)

- Absolute positional embedding: adding a fix (different embedding to each position of X):

  1. Add positional code:

$$X_0 := X_{oh}W_E^\intercal + P, \quad where\, P \in \mathbb{R}^{T \times d}$$

$$P := \begin{bmatrix} — & P_1^\intercal & — \\ — & P_2^\intercal & — \\ & \dots & \end{bmatrix}$$

     P_i are different, unique terms for each row (i.e. position in the input sequence).

2. Alternative: Relative positional embedding

$$\text{SelfAttention}(Q, K, V) := \text{softmax}\left(\frac{QK^\intercal}{\sqrt{d}} + D\right) V$$

where D encodes the distance between words in the sequence.

3. Alternative: Rotary positional embedding (RoPE), implemented in most modern LLMs

$$\text{SelfAttention}(Q, K, V) := \text{softmax}\left(\frac{QD'K^\intercal}{\sqrt{d}}\right) V$$

Masked Attention

- In a Transformer model, every output depends on every input:

$$\hat{Y} := \text{Transformer}(X_{oh})$$

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\intercal}{\sqrt{d}}\right) V =: AXW_V^\intercal$$

Applied to the input, we have:

$$\begin{bmatrix} — & a_1^\intercal & — \\ — & a_2^\intercal & — \\ & \vdots & \end{bmatrix} \begin{bmatrix} — & x_1^\intercal & — \\ — & x_2^\intercal & — \\ & \vdots & \end{bmatrix} = \begin{bmatrix} — & a_1^\intercal X & — \\ — & a_2^\intercal X & — \\ & \vdots & \end{bmatrix}$$

And every $a_i^\intercal X$ mixes parts from every token x_i in the input sequence.

- An alternative approach might be:

$$\begin{bmatrix} — & a_1^\intercal & — \\ — & a_2^\intercal & — \\ & \vdots & \end{bmatrix} \begin{bmatrix} — & x_1^\intercal & — \\ — & x_2^\intercal & — \\ & \vdots & \end{bmatrix} = \begin{bmatrix} — & \sum_{t=1}^{T} (a_1)_t x_t^\intercal & — \\ — & \sum_{t=1}^{T} (a_2)_t x_t^\intercal & — \\ & \vdots & \end{bmatrix}$$

If for each a_i only the elements with indices $< i$ were non-zero, we would avoid "looking back".

Leveraging the softmax operation, we can achieve that by letting:

$$\text{SelfAttention}(Q, K, V) := \text{softmax}\left(\frac{QK^\intercal + M}{\sqrt{d}}\right) V$$

where

$$M := \begin{bmatrix} 0 & -\infty & \dots & -\infty \\ 0 & 0 & \dots & -\infty \\ & \vdots & & \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

Since Q, K, and V are dependent on X, it is not obvious that this operation prevents all leakage. I'm not annotating the proof here, but it can be found in the video.

Lecture 13: Efficient LLM Inference

Generating text from an LLM

Sampling from an LLM:

- Take the last output $Y_T \in (R)^k$ from

$$Y := LLM(X_{\text{oh}}) = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_T \end{bmatrix}$$

- convert to "probability"

$$P := \text{softmax}(Y_T) = \frac{e^{Y_T}}{\sum_{j=1}^{T} e_i^Y}$$

- Sample next word from P, append to input sequence

```
import torch
batch_size = 1
seq_len = 100
k = 5
Y = torch.randn(batch_size, seq_len, k)
p = torch.softmax(Y[0, -1], dim=-1)
print(torch.cumsum(p, dim=-1))
torch.multinomial(p, 100, replacement=True) # sample 100 times
```

Temperature sampling

- Naive sampling: $P := \text{softmax}(Y_T)$
- Temperature sampling:

$$P := \text{softmax}\frac{Y_T}{\tau} \quad \tau \in \mathbb{R}_+$$

- Property: As $\tau \to 0$, P puts all weight on the most likely next word.
- Property: As $\tau \to \infty$, P becomes a uniform distribution.

KV Caching

- When we add a new row to out input, like:

$$X_{\text{oh}} = \begin{bmatrix} — & x_1^\intercal & — \\ — & x_2^\intercal & — \\ & \vdots & \\ — & x_T^\intercal & — \\ — & x_{T+1}2^\intercal & — \end{bmatrix} \mapsto Y = \begin{bmatrix} — & y_1^\intercal & — \\ — & y_2^\intercal & — \\ & \vdots & \\ — & y_T^\intercal & — \\ — & y_{T+1}2^\intercal & — \end{bmatrix}$$

1. $Y = \text{LLM}(X_{\text{oh}})$
2. $Y' = LLM(X'_{\text{oh}})$

- We want a way to calculate y_{T+1} as something like $\text{LLM}(x_{T+1})$.
- LLM Operation
  1. Right multiplications and non-linear operations
     Apply these independently to the rows of our internal representation, so they can be applied to a single additional row in isolation.
  2. Self Attention

     $$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\intercal}{\sqrt{d}}\right)V$$

     Looking at adding a new row:

     $$\begin{bmatrix} Y \\ y_{T+1}^\intercal \end{bmatrix} = \text{softmax}\left(\frac{\begin{bmatrix} Q \\ q_{T+1}^\intercal \end{bmatrix}\begin{bmatrix} K \\ k_{T+1}^\intercal \end{bmatrix}^\intercal}{\sqrt{d}}\right)\begin{bmatrix} V \\ v_{T+1}^\intercal \end{bmatrix}$$

- KV Caching Algorithm:
  1. Given x_{T+1} as new input to self attention
  2. Store K^{cache} and V^{cache}
  3. Form $q_{T+1} = W_Q x_{T+1}$, $k_{T+1} = W_K x_{T+1}$, $v_{T+1} = W_V x_{T+1}$
  4. Append to Cache:

     $$K^{\text{cache}} := \begin{bmatrix} K_{\text{old}}^{\text{cache}} \\ k_{T+1}^\intercal \end{bmatrix} \quad V^{\text{cache}} := \begin{bmatrix} V_{\text{old}}^{\text{cache}} \\ v_{T+1}^\intercal \end{bmatrix}$$

  5. Yield output:

     $$\begin{aligned} y_{T+1} &:= \text{SelfAttention}(q_{T+1}^\intercal, K^{\text{cache}}, V^{\text{cache}}) \\ &= \text{softmax}\left(\frac{q_{T+1}^\intercal K^{\text{cache}\intercal}}{\sqrt{d}}\right)V^{\text{cache}} \end{aligned}$$